Projektuppgift

Introduktion till webbutveckling med HTML, CSS och JavaScript

Julia Anderberg



MITTUNIVERSITETET
Institutionen för Informations-system och teknologi

Författare: Julia Anderberg, juan2602@student.miun.se
Utbildningsprogram: TWEUG, Webbutveckling, 120 hp
Huvudområde: Datateknik
Termin, år: HT, 2026

Julia Anderberg 2026-10-30
Projektuppgift

Sammanfattning

Projektet syftar till att utveckla en responsiv och användarvänlig webbplats för det fiktiva hantverksbageriet Écru Bageri. Webbplatsen presenterar bageriets sortiment och verksamhet samt göra relevant information lättillgänglig för besökaren.

Webbplatsen består av tre sidor: Startsida, Sortiment och Om oss. Besökaren kan utforska bageriets produkter, spara favoriter och ta del av information om bageriet. På sidan Om oss finns även kontaktuppgifter och ett kontaktformulär där besökaren kan lämna ett meddelande.

Innehållsförteckning

Sammanfattning iii

1 Introduktion 1
1.1 Bakgrund 2
1.2 Problemformulering 2

2 Teori 3

3 Metod 5

4 Konstruktion 7

5 Resultat 12

6 Diskussion / Slutsatser 15

Källförteckning 16

1 Introduktion

1.1 Bakgrund

En webbplats hjälper en verksamhet att göra sig synlig och nå ut till fler kunder. För ett bageri är det ett sätt att visa upp sitt sortiment, berätta om verksamheten och göra information om öppettider och kontaktuppgifter lättillgänglig. En tydlig webbplats kan göra det enklare för kunder att upptäcka bageriet och få en bild av vad det erbjuder. Mot denna bakgrund utvecklas en webbplats för det fiktiva hantverksbageriet Écru Bageri.

Écru Bageri är ett fiktivt nordiskt hantverksbageri som erbjuder vedugnsbakat bröd och rustika bakverk. Bageriet lägger stor vikt vid hantverket, bra råvaror och en enkel, tidlös stil. Målgruppen består främst av personer som uppskattar nybakat bröd och bakverk, värdesätter kvalitet och är intresserade av lokala verksamheter och närproducerade råvaror.

Webbplatsens syfte är att presentera bageriets sortiment och verksamhet på ett tydligt och lättillgängligt sätt. Besökare ska kunna läsa om bageriet, se vilka produkter som erbjuds och enkelt hitta kontaktuppgifter.

1.2 Problemmotivering och problemformulering

Struktur och navigering

Webbplatsen ska bestå av tre sammanlänkade sidor: en startsida, en sortimentssida och en sida om Écru Bageri. Varje sida ska ha ett tydligt syfte och innehålla relevant information för besökaren.

Navigeringen ska vara tydlig och likadan på samtliga sidor. I sidhuvudet på varje sida ska det finnas en gemensam navigeringsmeny med länkar till startsidan, sortimentssidan och sidan om Écru Bageri. Längst ner på varje sida ska det finnas en gemensam sidfot med bageriets adress och öppettider. På så sätt kan besökaren hitta denna information oavsett vilken sida som är öppen. På mindre skärmar ska menyn anpassas till en hamburgermeny som besökaren kan öppna och stänga för att visa eller dölja länkarna till sidorna.

Webbplatsens utseende och målgrupp

Webbplatsens utseende ska spegla Écru Bageris inriktning som ett nordiskt hantverksbageri. Designen ska anpassas efter målgruppen och förmedla kvalitet och tradition genom en enkel och tidlös stil, naturliga jordnära färger och bilder som framhäver bageriets produkter och hantverk.

Samtidigt ska webbplatsen vara enkel att använda och fungera på olika skärmstorlekar. En tydlig struktur och lättläst text ska göra innehållet lätt att läsa och förstå.

Sidorna

Startsidan ska presentera bageriet och dess inriktning. Den ska ge en kort beskrivning av verksamheten, visa ett urval av produkter som väcker intresse för sortimentet och innehålla en tydlig knapp som leder besökaren till sortimentssidan.

Sortimentssidan ska visa bageriets bröd och bakverk på ett tydligt sätt. Varje produkt ska presenteras med en bild, ett namn och en kort beskrivning. Besökaren ska kunna lägga till och ta bort bakverk från sin favoritlista genom att klicka på favoritknappen som finns vid varje produkt. Favoriterna ska sparas i webbläsarens localStorage och finnas kvar även när sidan laddas om.

Sidan Om Écru Bageri ska beskriva bageriets bakmetoder samt innehålla kontaktuppgifter. Besökaren ska enkelt kunna hitta bageriets e-postadress och telefonnummer. Sidan ska även ha ett kontaktformulär där besökaren kan ställa frågor eller skicka förfrågningar. Formuläret ska kontrollera att de obligatoriska fälten är ifyllda och visa tydliga felmeddelanden om uppgifter saknas eller har angetts i fel format.

Responsivitet och tillgänglighet

Webbplatsen ska byggas utifrån ett mobile-first-perspektiv, vilket innebär att layouten först anpassas för mindre skärmar och därefter för större skärmar. Text, bilder, navigering och produktpresentationer ska anpassas efter skärmstorleken så att webbplatsen är lättläst och enkel att använda på olika enheter.

Webbplatsen ska även uppfylla grundläggande krav på tillgänglighet. Innehållet ska vara tydligt uppdelat och följa en logisk ordning. Bilder ska ha beskrivande alternativtexter och formulärfält ska vara tydligt märkta. Detta gör det lättare för besökare att förstå innehållet och använda webbplatsens funktioner.

2 Teori

HTML, CSS och JavaScript

HTML (HyperText Markup Language) skapar en webbsidas innehåll och uppbyggnad [1]. CSS (Cascading Style Sheets) används för att bestämma hur sidan ska se ut visuellt, till exempel färger och textstorlek [2]. JavaScript är ett programmeringsspråk som används för att göra webbsidor interaktiva och styra hur de fungerar och reagerar på användarens handlingar [3]. Teknikerna samverkar genom att HTML används för att skapa innehållet på en webbsida, CSS bestämmer hur sidan ser ut och JavaScript gör att sidan kan reagera på användarens handlingar.

Responsiv webbdesign och CSS-layout

Vid utveckling av en webbplats med responsiv webbdesign anpassas sidans utformning efter skärmens storlek och den enhet som används. Syftet är att webbplatsen ska vara lätt att läsa och använda oavsett om webbplatsen besöks via en stor eller liten skärm. Detta görs bland annat med hjälp av CSS-regler som styr hur innehållet visas på olika skärmstorlekar [4].

Flexbox och CSS Grid är två sätt att ordna innehåll på en webbsida med CSS. Flexbox används för att placera element i en rad eller kolumn, medan CSS Grid används för att ordna innehåll i flera rader och kolumner. Vilket sätt som passar bäst beror på hur webbsidan ska utformas [5].

Media queries används för att anpassa CSS-regler efter exempelvis skärmens bredd. På så sätt kan webbplatsens layout ändras beroende på skärmstorlek, vilket är en viktig del av responsiv webbdesign [6].

DOM och händelsehantering

Document Object Model (DOM) är ett gränssnitt (API) som representerar ett HTML-dokument som en trädstruktur. Varje HTML-element motsvarar en nod (en del av dokumentets struktur) som JavaScript kan komma åt och ändra, exempelvis för att uppdatera innehåll [7].

Händelsehantering gör att en webbplats kan reagera på användarens handlingar, till exempel klicka på knappar. I JavaScript kan en funktion kopplas till en händelse och köras när den inträffar. Metoden addEventListener() används för att koppla en funktion till en viss händelse på ett HTML-element [8].

LocalStorage och formulärvalidering

Web Storage är ett webb-API som gör det möjligt att lagra data i användarens webbläsare. Det omfattar två lagringsmekanismer: localStorage och sessionStorage, som skiljer sig åt i hur länge data sparas. Med localStorage finns informationen kvar även efter att webbläsaren stängts och öppnats igen, tills den tas bort. Data lagras i form av nyckel–värde-par och hanteras med JavaScript. Funktionen kan exempelvis användas för att spara användarinställningar mellan besök [9].

Formulärvalidering innebär att kontrollera att användarens uppgifter uppfyller vissa krav, till exempel att obligatoriska fält är ifyllda och att e-postadressen har rätt format. JavaScript kan användas för att utföra kontrollerna och visa felmeddelanden [10]. Genom händelsehantering kan kontrollen utföras när användaren skickar formuläret. Metoden preventDefault() förhindrar då formulärets vanliga beteende, så att JavaScript kan kontrollera uppgifterna innan de skickas vidare [11].

Wireframe och mockup

En wireframe är en enkel skiss som visar hur innehållet på en webbsida ska placeras. Den visar sidans struktur och layout. En mockup är en mer detaljerad modell som visar hur webbplatsen ska se ut, till exempel vilka färger, bilder och typsnitt som ska användas [12].

3 Metod

Planering och design

I planeringsfasen användes wireframes för att bestämma webbplatsens struktur och layout innan arbetet med kodningen började. En wireframe skapades för startsidan i mobilformat och tre för större skärmar, en för varje sida: Startsida, Sortiment och Om oss. Det skapades även en mockup av startsidan för större skärmar för att planera mer detaljerade färger, typsnitt, bilder och andra delar av webbplatsens utseende.

En wireframe för mindre skärmar skapades endast för startsidan eftersom webbplatsens tre sidor har flera gemensamma delar, bland annat header och footer. Även sidornas huvuddelar liknar varandra i layouten. Därför ansågs det inte nödvändigt att skapa en separat mobilwireframe för varje sida. I stället användes startsidans wireframe som grund för hur innehållet på samtliga sidor skulle anpassas till mindre skärmar.

Tekniker för layout och responsiv design

För webbplatsens layout användes Flexbox och CSS Grid. Flexbox användes för att placera innehåll i rader eller kolumner, exempelvis i sidhuvudet och sidfoten. CSS Grid användes för att ordna produktkort och andra sektioner i flera kolumner. Valet av layoutteknik anpassades efter hur innehållet behövde placeras på respektive del av webbplatsen.

För att webbplatsen skulle fungera på både mindre och större skärmar användes en mobile-first-design med CSS media queries för att anpassa layouten efter skärmens storlek.
En brytpunkt på 768 pixlar valdes för att anpassa layouten till större skärmar. Vid denna bredd visas vissa delar av innehållet i flera kolumner bredvid varandra i stället för att placeras under varandra. Det gör att skärmutrymmet används bättre och att innehållet förblir tydligt och lättläst.

Val av JavaScript tekniker

För att anpassa navigeringen till mindre skärmar används en hamburgermeny. JavaScript gör att menyn öppnas och stängs när användaren trycker på menyknappen. Det gör det enklare att navigera på webbplatsen när skärmutrymmet är begränsat.

JavaScript används också för att validera uppgifterna i kontaktformuläret. Kontrollen ser till att obligatoriska fält är ifyllda och att e-postadressen har rätt format. Om uppgifter saknas eller är felaktiga visas felmeddelanden som hjälper användaren att rätta till dem.

På sortimentssidan finns en funktion för att markera och avmarkera produkter som favoriter. Favoriterna sparas i webbläsarens localStorage, vilket gör att markeringarna finns kvar även när sidan laddas om. Användaren kan därmed återvända till sortimentet och se sina tidigare val.

Julia Anderberg 2026-10-30
Projektuppgift

Versionshantering och publicering

För att versionshantera projektet användes Git och GitHub via Visual Studio Code. Ändringar i projektets filer sparades regelbundet genom beskrivande commits och skickades till GitHub med push. Projektet delades upp i två grenar: dev och main. Utvecklingsarbetet genomfördes i dev, medan main användes för den färdiga versionen av webbplatsen. När utvecklingen var klar fördes ändringarna från dev över till main genom en merge.

Webbplatsen publicerades med GitHub Pages.

Testning

För att kontrollera att webbplatsen fungerade som avsett genomfördes manuella tester. Syftet var att kontrollera att funktionerna fungerade och att innehållet visades tydligt på olika skärmstorlekar.

Navigeringen testades genom att öppna de olika sidorna och kontrollera att länkarna fungerade. Hamburgermenyn testades genom att öppna och stänga den på mindre skärmar. Favoritfunktionen testades genom att lägga till och ta bort favoriter samt kontrollera att markeringarna fanns kvar efter att sidan laddats om. I webbläsarens utvecklarverktyg kontrollerades också att favoritmarkerade produkter sparades i arrayen i localStorage. Kontaktformuläret testades med både korrekta och felaktiga uppgifter för att kontrollera att rätt felmeddelanden visades.

Webbplatsen testades vid olika skärmstorlekar för att kontrollera att innehållet anpassades till skärmens bredd och att navigering, text, bilder och produktkort var tydliga och fungerade som avsett. JavaScript-koden kontrollerades även i webbläsarens konsol för att upptäcka eventuella fel.

HTML- och CSS-koden kontrollerades med W3C valideringsverktyg för att hitta eventuella kodfel.

4 Konstruktion

Webbplatsens utseende

Innan arbetet med koden påbörjades gjordes wireframes och en mockup i Figma för att planera webbplatsens struktur och utseende. Alla sidor delar samma sidhuvud med en klickbar logotyp som leder till startsidan och en navigeringsmeny. De har även en gemensam sidfot med adress och öppettider. Följande avsnitt beskriver innehållet i varje sidas <main>-element.

Bilderna på webbplatsen hämtades från Unsplash och valdes ut för att passa bageriets estetik och visa produkterna.



Figur 1:1 Wireframe för mindre skärm

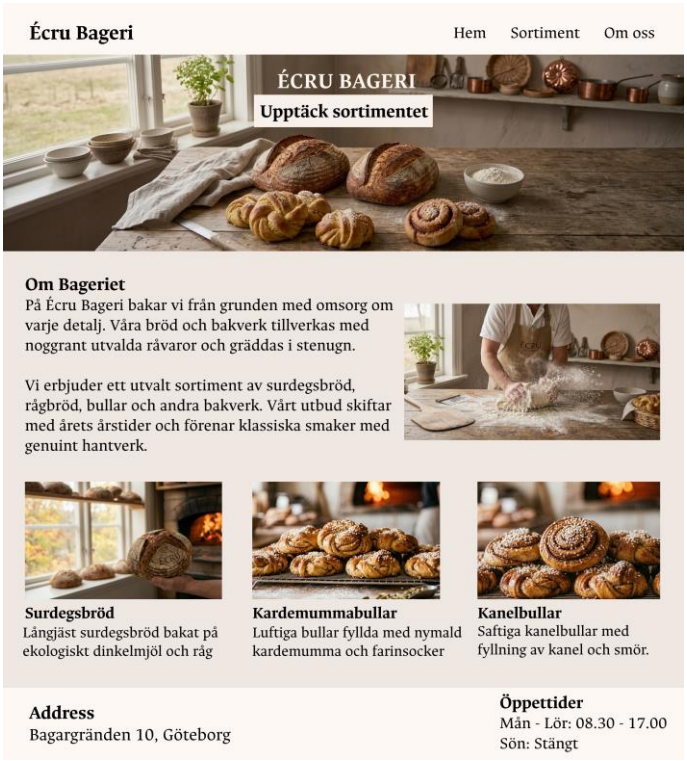
Arbetet utgick från en mobile-first-design, där layouten för mindre skärmar planerades först och därefter anpassades för större skärmar. Som visas i Figur 1:1 staplas innehållet i en

kolumn för att underlätta läsningen, medan navigeringen ersätts av en hamburgermeny för att spara utrymme.



Figur 1:2 Wireframes för större skärm

Figur 1:2 visar wireframes för webbplatsens tre sidor på större skärmar. Innehållet är uppdelat i flera kolumner placerade brevid varandra, exempelvis produktkorten på startsidan och sortimentssidan. Det gör att att innehållet blir lättare att överblicka.



Figur 1:3 Mockup startsidan för större skärmar

För att planera startsidans utseende skapades en mockup, som visas i figur 1:3. Färger i varma, jordnära toner och fotografier valdes för att passa bageriets hantverksmässiga profil. Rubrikerna använder typsnittet Serif, medan brödtexten använder typsnittet Georgia för att vara lättläst. Vid kodningen gjordes några ändringar jämfört med mockupen. Knappen flyttades till en plats under hero-bilden för att göra den tydligare och lättare att hitta. Bilden i

sidhuvudet byttes också ut eftersom den tog för mycket uppmärksamhet från huvudrubriken, som därför blev svårare att läsa.

Responsiv design

Webbplatsen anpassades för olika skärmstorlekar med CSS enligt mobile-first-principen. På mindre skärmar placeras innehållet huvudsakligen i en kolumn med hjälp av Flexbox. För större skärmar används en media query med brytpunkten 768 pixlar. Vid brytpunkten ändras layouten så att innehållet placeras i flera kolumner bredvid varandra i stället för att staplas vertikalt. Detta görs med CSS Grid.

Navigeringen anpassas efter skärmstorleken. På mindre skärmar döljs navigeringslänkarna och ersätts av en hamburgermeny för att ta mindre plats. På större skärmar visas istället länkarna i en horisontell rad i sidhuvudet. Logotypen i sidhuvudet fungerar också som en länk till startsidan, så att besökaren enkelt kan gå tillbaka dit från de andra sidorna. Sidfoten anpassas på samma sätt och visas vertikalt på mindre skärmar och horisontellt på större skärmar.

Interaktivitet genom JavaScript

Hamburgermeny

Hamburgermenyn skapades med JavaScript genom att koppla en händelselyssnare till menyknappen. När användaren klickar på knappen öppnas eller stängs menyn genom att CSS-klassen open läggs till eller tas bort från navigeringselementet. Funktionen uppdaterar även attributet aria-expanded för att ange om menyn är öppen eller stängd.

Menyknappen skapades i HTML med ett <button>-element som innehåller tre -element. Med CSS formas dessa till hamburgersymbolen. Knappen har även attributet aria-label="Meny", som anger vad knappen används till för hjälpmedel som skärmläsare.

```
function toggleMenu() {
    siteNav.classList.toggle("open");

    const menuIsOpen = siteNav.classList.contains("open");

    hamburgerMenu.setAttribute("aria-expanded", menuIsOpen);
}
```

Figur 2:1 JavaScript-kod för att öppna / stänga hamburgermenyn

Som visas i figur 2:1 används funktionen toggleMenu() för att öppna och stänga navigeringsmenyn. Funktionen lägger till CSS-klassen open om den inte redan finns och tar bort den om den finns. CSS-koden avgör sedan om menyn ska visas. Därefter kontrollerar classList.contains("open") om klassen finns och uppdaterar attributet aria-expanded utifrån detta. Attributet gör det möjligt för hjälpmedel, exempelvis skärmläsare, att ange om menyn är öppen eller stängd.

Validering av kontaktformulär

JavaScript används också för att kontrollera uppgifterna i kontaktformuläret på sidan Om oss. När formuläret skickas körs funktionen validateForm(), som kontrollerar att de obligatoriska fälten är ifyllda och att e-postadressen har rätt format. Om något fält inte är korrekt ifyllt visas ett felmeddelande vid det specifika fältet.

En array med namnet errors används för att samla fälten som innehåller fel. Funktionen returnerar true om inga fel hittas och false om minst ett fel finns. Med event.preventDefault() förhindras formuläret från att skickas på vanligt sätt.

Problem och lösning - Kontaktformulär

```
// Körs när kontaktformuläret skickas
if (contactForm) {
contactForm.addEventListener("submit", function (event) {
    event.preventDefault();

// Validera inmatningen
validateForm();

});
}
```

Figur 2:2 JavaScript-kod som kontrollerar att kontaktformuläret finns innan en händelselyssnare kopplas till det

Eftersom samma JavaScript-fil används på alla sidor uppstod ett problem när koden skulle kopplas till kontaktformuläret, som endast finns på sidan Om oss. På de andra sidorna saknas formuläret, vilket gjorde att koden försökte komma åt ett element som inte fanns. Problemet löstes genom att lägga till en if-kontroll, som visas i figur 2:2. Kontrollen undersöker om kontaktformuläret finns på sidan innan en händelselyssnare kopplas till det. Därmed kan samma JavaScript-fil användas på samtliga sidor utan att orsaka fel.

```
// Kontrollerar att e-postadressen är ifylld
if (email === "") {
    errors.push("email");
    emailError.textContent = "Fyll i din e-postadress.";

// Kontrollerar att e-postadressen har ett rimligt format
} else if (emailInput.validity.typeMismatch) {
    errors.push("email");
    emailError.textContent = "Ange en giltig e-postadress.";

}
```

Figur 2:3 Validering av e-post i kontaktformuläret

I figur 2:3 visas hur e-postadressen kontrolleras i två steg. Först kontrolleras om e-postfältet är tomt. Om det är tomt läggs värdet "email" till i arrayen errors och ett felmeddelande visas. Om fältet innehåller text kontrolleras om e-postadressen har rätt format med hjälp av emailInput.validity.typeMismatch. Om formatet är felaktigt läggs även då värdet "email" till i arrayen och ett felmeddelande visas. På så sätt får användaren veta varför e-postadressen inte godkänns.

Favoritprodukter

På sortimentssidan finns en funktion som gör det möjligt att lägga till och ta bort produkter från en favoritlista. Varje favoritknapp är kopplad till en produkt genom attributet data-product. När användaren klickar på knappen läggs produkten till i favoritlistan eller tas bort om den redan finns där. För att favoriterna ska finnas kvar även efter att sidan laddas om sparas listan i webbläsarens localStorage.

```
// Sparar favoriterna i localstorage
function saveFavorites() {
    localStorage.setItem("favorites", JSON.stringify(favorites));
}

// Läser in tidigare sparade favoriter
function loadFavorites() {
    const savedFavorites = localStorage.getItem("favorites");

    if (savedFavorites) {
        favorites = JSON.parse(savedFavorites);
    }
}
```

Figur 2:4. JavaScript-kod för att spara och hämta favoritprodukter

Som visas i figur 2:4 används funktionen saveFavorites() för att spara favoritlistan och loadFavorites() för att hämta sparade favoriter när sidan laddas. Med JSON.stringify() omvandlas favoritlistan till en text som kan sparas i localStorage. När sidan laddas hämtas informationen med localStorage.getItem() och omvandlas tillbaka till en array med JSON.parse(). Därefter uppdaterar renderFavorites() favoritknapparna så att de visar vilka produkter som är sparade som favoriter.

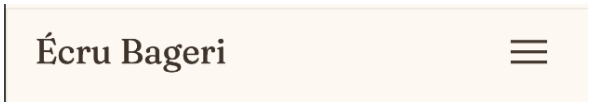
5 Resultat

Webbplatsens uppbyggnad

Projektet resulterade i en webbplats för det fiktiva hantverksbageriet Écru Bageri. Webbplatsen har tre sidor: Startsida, Sortiment och Om oss. Alla sidor har samma meny och sidfot samt en gemensam estetik med jordnära färger.

På startsidan får besökaren en kort presentation av bageriet och kan se några av produkterna. På sortimentssidan visas sex produkter med bilder, namn och korta beskrivningar. Där kan besökaren också lägga till och ta bort produkter som favoriter. På sidan På sidan Om oss presenteras bageriets bakningsprocess och innehåller även kontaktuppgifter, samt ett kontaktformulär.

Webbplatsens layout anpassas efter skärmstorleken. På mindre skärmar visas innehållet främst i en kolumn och navigeringen sker genom en hamburgermeny. På större skärmar visas navigeringslänkarna bredvid varandra och vissa delar av innehållet delas upp i flera kolumner.



Figur 2:5 Sidhuvud med hamburgermeny på mindre skärm



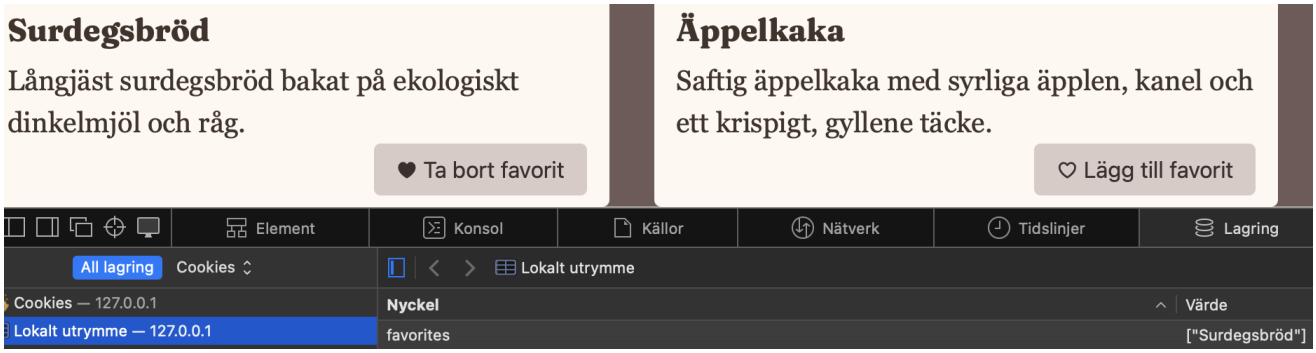
Figur 2:6 Utfälld hamburgermeny på mindre skärm

Figur 2:5 visar hamburgermenyn i stängt läge på en mindre skärm, med menyknappen placerad i sidhuvudet. I figur 2:6 visas menyn när den är öppnad. Då visas navigeringslänkarna över hela skärmens bredd, med en länk på varje rad och texten centrerad. För att skilja länkarna åt har CSS-egenskapen border-bottom använts, vilket lägger till en tunn linje under varje länk. Med selektorn :not(:last-child) anges att alla länkar utom den sista ska ha en linje under sig. På så sätt undviks en extra linje längst ned i menyn.



Figur 2:7 Kontaktformulär med felmeddelanden vid saknade / felaktiga uppgifter

Figur 2:7 visar kontaktformuläret på sidan Om oss. Namn- och meddelandefälten är tomma och e-postadressen har fel format. Formuläret visar alltså felmeddelanden vid fälten för att tydliggöra vilka uppgifter som behöver fyllas i eller rättas.



Figur 2:8 Favoritmarkerad produkt och sparad favorit i webbläsarens localStorage

Figur 2:8 visar favoritfunktionen på sortimentssidan. Surdegsbröd är markerat som favorit, medan äppelkaka inte är det. I webbläsarens utvecklarverktyg visas att surdegsbröd har sparats i localStorage. Favoritmarkeringen finns kvar efter att sidan laddas om, vilket betyder att funktionen fungerar som avsett.

Resultat från tester

Tabellen visar resultaten från genomförda tester och kontroller. Den visar i vilken utsträckning webbplatsen uppfyller de krav som ställdes i början av projektet.

Test	Ja/Nej
Navigeringen fungerar mellan samtliga tre sidor	**Ja**
Hamburgermenyn fungerar på mindre skärmar	Ja
Favoritprodukter kan läggas till och tas bort	Ja
Favoriter sparas efter omladdning av sidan	Ja
Kontaktformuläret visar felmeddelanden vid tomma obligatoriska fält	Ja
Kontaktformuläret validerar e-postadressens format	Ja
Webbplatsens layout anpassas till olika skärmstorlekar	Ja
HTML-och CSS kod valideras korrekt genom W3C-validator	Ja
Semantiska HTML-element används för att strukturera innehållet	Ja
Bilder har relevanta alternativa texter	Ja

6 Diskussion / Slutsatser

Målet med projektet var att skapa en responsiv och användarvänlig webbplats för det fiktiva hantverksbageriet Écru Bageri. Jag tycker att målet till stor del har uppnåtts eftersom webbplatsen innehåller de planerade sidorna och funktionerna samt ger besökaren möjlighet att ta del av bageriets produkter och kontaktuppgifter. Testerna visar att navigeringen, hamburgermenyn, favoritfunktionen och kontaktformuläret fungerar som de ska. HTML- och CSS-koden har också kontrollerats med W3C valideringsverktyg utan att några fel upptäcktes.

Under arbetet stötte jag på några tekniska problem som behövde felsökas. Ett exempel var att kontaktformuläret bara fanns på sidan Om oss, medan samma JavaScript-fil användes på alla sidor. Det gjorde att koden försökte hitta ett formulär även på sidor där det inte fanns något. Jag löste detta genom att lägga till en kontroll som undersöker om kontaktformuläret finns på sidan innan formulärkoden körs. Detta känner jag gav mig en bättre förståelse för hur JavaScript-kod behöver anpassas efter sidornas innehåll.

Webbplatsens utseende behövde också ändras från den ursprungliga mockupen för att skapa en tydligare struktur och fungera bättre på olika skärmstorlekar. Arbetet har gett mig mer kunskap om hur design och funktion hänger ihop och hur val av exempelvis placering av länkar, text och knappar påverkar hur lätt webbplatsen är att använda.

Om jag skulle förbättra något med webbplatsen hade jag skapat en separat kontaktsida istället för att placera kontaktuppgifterna och formuläret på sidan Om oss. Det hade gjort det något enklare för besökaren att hitta kontaktinformationen.

Källförteckning

[1] MDN Web Docs. *HTML: HyperText Markup Language* https://developer.mozilla.org/en-US/docs/Web/HTML [Hämtad22 oktober 2026]

[2] MDN Web Docs. *CSS: Cascading Style Sheets* https://developer.mozilla.org/en-US/docs/Web/CSS [Hämtad 22 oktober 2026]

[3] MDN Web Docs. *JavaScript*. https://developer.mozilla.org/en-US/docs/Web/JavaScript [Hämtad 22 oktober 2026]

[4] Mittuniversitetet (MIUN) [2026] *CSS – Responsivitet, layout och positionering* Föreläsningsmaterial [Hämtad 26 oktober 2026]

[5] Mittuniversitetet (MIUN) [2026] *CSS – Responsivitet, layout och positionering* Föreläsningsmaterial [Hämtad 26 oktober 2026]

[6] MDN Web Docs. *Media queries* https://developer.mozilla.org/en-US/docs/Web/CSS/Guides/Media_queries [Hämtad 26 oktober 2026]

[7] Mittuniversitetet (MIUN) [2026] *Moment 5: DOM (Document Object Model)* Kursmaterial på lärplattformen [Hämtad 26 oktober 2026]

[8] Mittuniversitetet (MIUN) [2026] *Moment 5: Events (händelsehantering)* Kursmaterial på lärplattformen [Hämtad 26 oktober 2026]

[9] Mittuniversitetet (MIUN) [2026] *Moment 5: Web Storage och cookies*. Kursmaterial på lärplattformen [Hämtad 26 oktober 2026]

[10] Mittuniversitetet (MIUN) [2026] *Moment 5: Formulärhantering* Kursmaterial på lärplattformen [Hämtad 26 oktober 2026]

[11] Mittuniversitetet (MIUN) [2026] *Moment 5: Validering och felhantering* Kursmaterial på lärplattformen [Hämtad 26 oktober 2026]

[12] Mittuniversitetet (MIUN) [2026] *Moment 3: Designprocess – wireframes, mockups och prototyper* Kursmaterial på lärplattformen [Hämtad 30 oktober 2026]